

GitHub y Repositorios Remotos

Repositorios remotos, autenticación, trabajo colaborativo

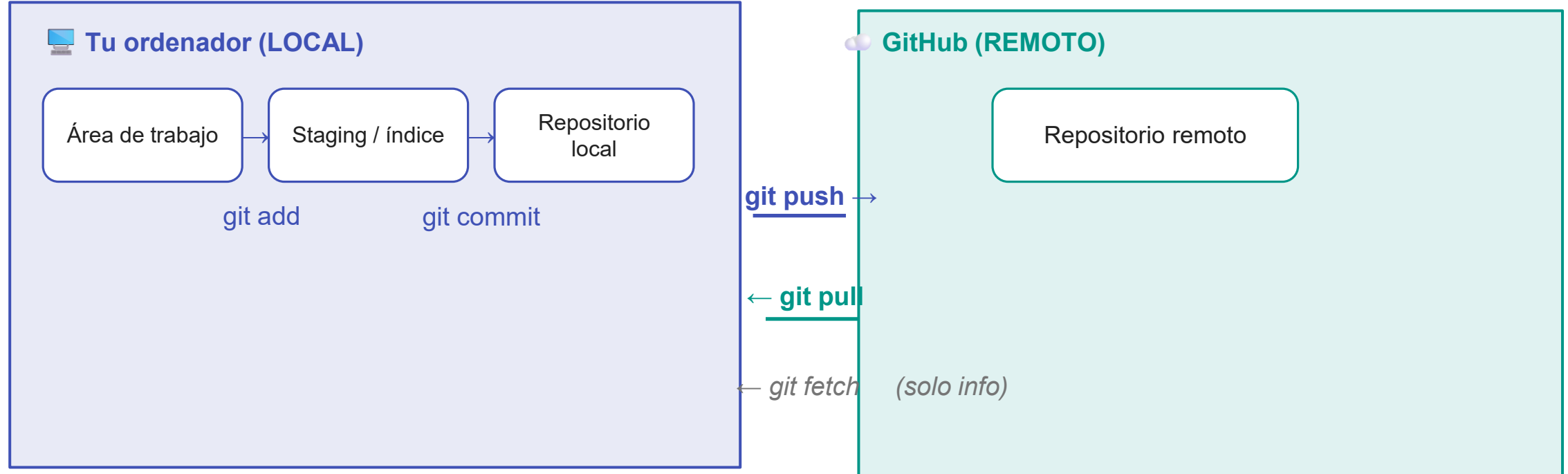
¿Qué es un Repositorio Remoto?

Un repositorio remoto es una copia de tu repositorio de Git alojada en un servidor de internet. Contiene una réplica completa de la historia del proyecto: commits, ramas y etiquetas.

Principales utilidades

- **Colaboración**
 - Varios desarrolladores pueden trabajar sobre el mismo proyecto y compartir sus cambios.
- **Copia de seguridad**
 - Si tu ordenador falla, el código sigue estando en la nube.
- **Distribución**
 - Puedes mostrar tu trabajo como portafolio o conectarlo a servidores de despliegue.

Flujo local ↔ remoto



Alternativas a GitHub: • GitLab (open source, muy usado en empresas) • Bitbucket (Atlassian, integra con Jira/Trello)

Registro en GitHub

Antes de crear repositorios remotos o colaborar en proyectos, necesitas una cuenta. El proceso es gratuito.

Pasos para registrarse

- Visita <https://github.com/> y haz clic en “Sign up” (arriba a la derecha).
- Introduce tu correo electrónico, elige una contraseña y un nombre de usuario.
- Resuelve el puzzle de verificación.
- GitHub te enviará un código numérico al correo. Intróduclo en la pantalla de confirmación.
- Ya tienes acceso a tu panel de control (dashboard) desde donde puedes crear y explorar repositorios.

Sign up for GitHub

 Continue with Google

 Continue with Apple

or

Email*

Email

Password*

Password

Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.

Username*

Username

Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.

Your Country/Region*

Spain

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

Email preferences

☐ Receive occasional product updates and announcements

Create account >

Dashboard de GitHub tras iniciar sesión

Panel de control de GitHub: a la izquierda el listado de repositorios (vacío al empezar) y el botón verde “New” para crear uno nuevo.

The screenshot shows the GitHub Dashboard interface. At the top, there's a navigation bar with the GitHub logo, the word "Dashboard", a search bar with the placeholder "Type / to search", and several utility icons (notifications, plus, clock, pull requests, repository, and profile). On the left sidebar, under "Top repositories", there's a "New" button and a search bar "Find a repository...". Below this is a list of repositories by "aitorventura": "entornos-de-desarrollo", "gatos", "otra-prueba-git", "proyecto-git", "sostenibilidad", "proyecto-ramas", and "actividades-programacion". A "Show more" link is at the bottom of the list. The main content area is titled "Home". It features a large "Ask anything" section with a search bar and buttons for "Ask", "All repositories", and a plus icon. Below this are several action buttons: "Task", "Create issue", "Write code", "Git", and "Pull requests". At the bottom, there's a "Feed" section with a "Filter" button and a "Trending repositories" link with a "See more" link.

Dashboard

Find a repository...

aitorventura/entornos-de-desarrollo

aitorventura/gatos

aitorventura/otra-prueba-git

aitorventura/proyecto-git

aitorventura/sostenibilidad

aitorventura/proyecto-ramas

aitorventura/actividades-programacion

Show more

Home

Ask anything

Ask All repositories +

Task Create issue Write code Git Pull requests

Feed

Filter

Trending repositories · [See more](#)

Autenticación y Conexión en GitHub

Desde la terminal no se puede usar la contraseña de la cuenta. GitHub exige uno de estos métodos:

Método	Cómo funciona	Cuándo usarlo
Token PAT	Generas una “contraseña larga” específica para repositorios, revocable en cualquier momento.	Método más habitual al empezar.
Claves SSH	Par de claves criptográficas en tu ordenador. Una vez configuradas, no pide credenciales.	Método profesional, más cómodo a largo plazo.
GitHub CLI (gh)	Herramienta oficial de consola que autentica a través del navegador.	Cuando se trabaja mucho con la CLI de GitHub.

 **Recomendación:** El PAT es más rápido de configurar al principio. Las claves SSH son más seguras y no requieren pegar ningún token en cada push. En un entorno profesional, SSH es el estándar.

Generar un Token de Acceso Personal (PAT)

Pasos en GitHub

- En GitHub, haz clic en tu foto de perfil → Settings.
- Baja por el menú lateral hasta < > Developer settings.
- Ve a Personal access tokens → Tokens (classic).
- Pulsa Generate new token → Generate new token (classic).
- Asigna un nombre descriptivo y marca la casilla “repo” para permisos de lectura/escritura.
- Baja al final de la página y haz clic en Generate token.

 Esta es la única vez que GitHub te muestra el token. Si cierras la pestaña sin copiarlo, tendrás que generar uno nuevo.

Formulario de creación de Token PAT

El formulario permite asignar nombre, vencimiento y permisos. Marca la casilla “repo” para dar acceso de lectura y escritura sobre repositorios.

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

Token para DAW

What's this token for?

Expiration

 30 days (Mar 23, 2026) ▾

The token will expire on the selected date

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

- | | |
|---|--------------------------------------|
| ☒ repo | Full control of private repositories |
| ☒ repo:status | Access commit status |
| ☒ repo_deployment | Access deployment status |
| ☒ public_repo | Access public repositories |
| ☒ repo:invite | Access repository invitations |
| ☒ security_events | Read and write security events |

Token generado — cópialo ahora

⚠ Copia el token completo ahora. GitHub no volverá a mostrártelo. Cuando Git pida contraseña al hacer push, pega este token.

Personal access tokens (classic)

Generate new token ▼

Tokens you have generated that can be used to access the [GitHub API](#).



Make sure to copy your personal access token now. You won't be able to see it again!



ghp_

[Redacted token]



Delete

Configurar SSH — Paso 1: Generar la clave

Si no quieres pegar el token cada vez que subes código, configura una clave SSH. El proceso es de una sola vez.

Genera la clave en Git Bash con tu correo de GitHub

```
ssh-keygen -t ed25519 -C "tu_correo@ejemplo.com"
```

- La terminal pregunta dónde guardar la clave: pulsa Enter para aceptar la ruta por defecto.
- Puede pedir una passphrase (contraseña de la clave): pulsa Enter para dejarlo en blanco.
- El dibujo ASCII al final confirma que la clave se ha generado en la carpeta .ssh de tu ordenador.

SSH — Terminal generando la clave

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/apuntes-profe/entornos-de-desarrollo (main)
```

```
$ ssh-keygen -t ed25519 -C aitor[REDACTED]@gmail.com
```

```
Your identification has been saved in /c/Users/Aitor/.ssh/id_ed25519
```

```
Your public key has been saved in /c/Users/Aitor/.ssh/id_ed25519.pub
```

```
The key fingerprint is:
```

```
SHA256:R8ayZTb8Dtj+aDFGXYDTbXkhR9OzhG70iNNDJ6qxf0s aitor[REDACTED]@gmail.com
```

```
The key's randomart image is:
```

```
+--[ED25519 256]--+
```

```
|      o.+o*o|
```

```
| [REDACTED]|
```

```
| . XoBo*.o|
```

```
| %.*.* o|
```

```
| [REDACTED]|
```

```
| =+o|
```

```
| .oo.E|
```

```
| [REDACTED]|
```

```
| .. o..|
```

```
+-----[SHA256]-----+
```

Configurar SSH — Paso 2: Copiar la clave pública

El proceso ha generado dos archivos: la clave privada (nunca la compartas) y la clave pública (la que le damos a GitHub).
Copia la pública:

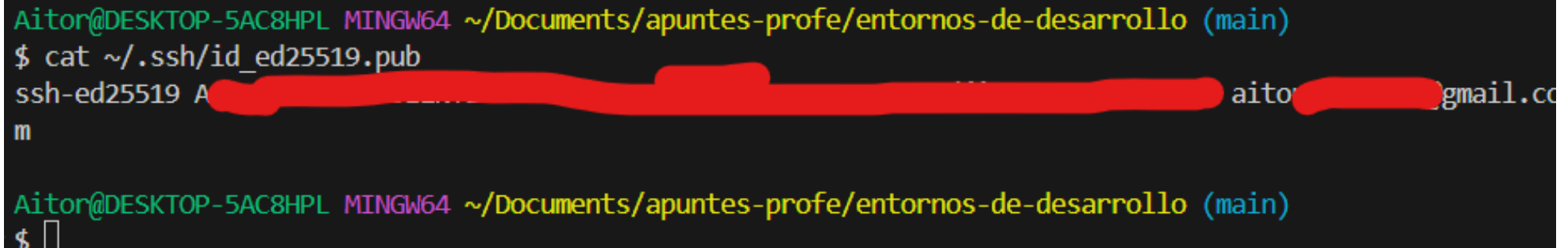
Comandos para copiar

```
# Windows (Git Bash)
clip < ~/.ssh/id_ed25519.pub

# Mac
pbcopy < ~/.ssh/id_ed25519.pub

# O muéstrala en pantalla y cópiala manualmente:
cat ~/.ssh/id_ed25519.pub
```

- La clave pública siempre empieza por ssh-ed25519.
- La clave privada (sin .pub) no debes compartirla nunca con nadie.



```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/apuntes-profe/entornos-de-desarrollo (main)
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 A[REDACTED] aitor[REDACTED]@gmail.co
m

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/apuntes-profe/entornos-de-desarrollo (main)
$
```

Configurar SSH — Paso 3: Añadir la clave a GitHub

Pasos en la web de GitHub

- Ve a GitHub → tu foto de perfil → Settings → SSH and GPG keys.
- Pulsa New SSH key.
- Escribe un título (p. ej. “Portátil clase”) y pega la clave pública en el campo de texto.
- Haz clic en Add SSH key.

A partir de ahora, cuando clones un proyecto usa la URL SSH (empieza por `git@github.com:...`) en lugar de HTTPS. Cualquier push o pull funcionará sin pedirte credenciales.

Add new SSH Key

Title

Mi ordenador

Key type

Authentication Key ↕

Key

ssh-ed25519 A[redacted] aitor[redacted]@gmail.com

Add SSH key

SSH — Clave añadida correctamente

SSH keys

[New SSH key](#)

This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.

Authentication keys



Mi ordenador

SHA256: [redacted]

Added on Feb 21, 2026

Never used — Read/write

SSH

[Delete](#)

Check out our guide to [connecting to GitHub using SSH keys](#) or troubleshoot [common SSH problems](#).

Subir tu proyecto — Paso 1: Crear el repositorio en GitHub

Imagina que has estado trabajando en local con `git init`, `git add` y `git commit`. Ahora quieres subirlo a GitHub.

Crear un repositorio vacío

- Entra en tu cuenta de GitHub y pulsa el botón New para crear un nuevo repositorio.
- Dale un nombre y decide si será público o privado.
- **IMPORTANTE: Si ya tienes un repositorio local listo, NO marques las opciones de añadir README, .gitignore ni licencia desde la web.**
- Necesitas que GitHub te dé un repositorio completamente vacío para que no haya conflictos con tu historial local.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 **General**

Owner * aitorventura / Repository name *

mi_proyecto is available.

Great repository names are short and memorable. How about [bug-free-octo-disco](#)?

Description

0 / 350 characters

2 **Configuration**

Choose visibility * Public
Choose who can see and commit to this repository

Add README Off
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license
Licenses explain how others can use your code. [About licenses](#)

[Create repository](#)

Paso 2: Enlazar local con remoto (git remote add)

El alias “origin” es un apodo que le damos a la URL del servidor remoto. Por convención siempre se llama origin; así no tienes que escribir la URL completa cada vez.

Según el protocolo configurado

HTTPS:

```
git remote add origin https://github.com/tu-usuario/tu-repositorio.git
```

SSH:

```
git remote add origin git@github.com:tu-usuario/tu-repositorio.git
```

Verificar que el enlace se ha creado:

```
git remote -v
```


git remote -v — origin enlazado

git remote -v confirma que origin apunta a la URL correcta tanto para push como para fetch.

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/mi_proyecto (main)
• $ git remote add origin https://github.com/aitorventura/mi_proyecto.git

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/mi_proyecto (main)
• $ git remote -v
origin https://github.com/aitorventura/mi_proyecto.git (fetch)
origin https://github.com/aitorventura/mi_proyecto.git (push)

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/mi_proyecto (main)
○ $
```

Paso 3: El primer push (git push -u origin main)

La primera vez añade -u (--set-upstream) para vincular tu rama local con la rama remota. A partir de ahí, Git sabe adónde ir cuando ejecutas git push o git pull a secas.

```
git push -u origin main
```

Info: ¿Por qué me pide (o no me pide) contraseña?

- **Si pide usuario y contraseña:**
 - Pega el Token PAT como contraseña. Tu contraseña de GitHub no funcionará.
- **Si no pide nada:**
 - Tienes SSH configurado, o el Git Credential Manager de Windows ha guardado tus credenciales.

⚠ No uses git push --force salvo que estés seguro. --force sobrescribe el historial del servidor y puede borrar el trabajo de tus compañeros. Haz primero git pull y luego vuelve a hacer push.

Primer push exitoso

git push -u origin main con las estadísticas de éxito (Writing objects: 100%) y el aviso de que la rama remota queda enlazada.

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/mi_proyecto (main)
$ ls
primer_doc.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/mi_proyecto (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 213 bytes | 213.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/aitorventura/mi_proyecto.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/mi_proyecto (main)
$
```

Repositorio en GitHub con los archivos subidos

El repositorio en GitHub muestra los archivos locales junto al mensaje y la fecha de cada commit.

 **mi_proyecto** Public

 Pin  Watch 0

 main  1 Branch  0 Tags



 Add file  Code

 **aitorventura** Primer doc d5d381e · 5 minutes ago  1 Commit

 primer_doc.txt Primer doc 5 minutes ago

Clonar un repositorio (git clone)

Llegas a un ordenador donde no tienes nada del proyecto. En lugar de git init, usas git clone con la URL del repositorio.

```
git clone https://github.com/tu-usuario/tu-repositorio.git
# O con SSH si lo tienes configurado:
git clone git@github.com:tu-usuario/tu-repositorio.git
```

git clone hace tres cosas de golpe

- Crea una carpeta con el nombre del repositorio.
- Descarga todos los commits, ramas y archivos.
- Configura origin apuntando al remoto automáticamente.

Para obtener la URL: entra en el repositorio y haz clic en el botón verde “<> Code”.

Botón Code — cómo obtener la URL de clonación

The screenshot shows the GitHub interface for a repository named 'mi_proyecto' by user 'aitorventura'. The repository is public and has 1 branch and 0 tags. The 'main' branch is selected. A search bar 'Go to file' and buttons 'Add file' and '<> Code' are visible. The 'Code' button is open, showing options for cloning the repository. The 'Local' tab is selected, and the 'Clone' button is highlighted. The cloning URL is 'git@github.com:aitorventura/mi_proyecto.git'. Below the URL, there is a note 'Use a password-protected SSH key.' and two other options: 'Open with GitHub Desktop' and 'Download ZIP'.

mi_proyecto Public

Pin Watch 0

main 1 Branch 0 Tags

Go to file t Add file <> Code

aitorventura Primer doc

primer_doc.txt Primer doc

README

Add a README

Help people interested in this repository

Add a README

Local Codespaces

Clone ?

HTTPS SSH GitHub CLI

git@github.com:aitorventura/mi_proyecto.git

Use a password-protected SSH key.

Open with GitHub Desktop

Download ZIP

git clone en la terminal

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/otro_directorio (main)
$ git clone https://github.com/aitorventura/mi_proyecto.git
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/otro_directorio (main)
$ cd mi_proyecto/

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/otro_directorio/mi_proyecto (main)
$ ls
primer_doc.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/otro_directorio/mi_proyecto (main)
$
```

git fetch vs git pull

	git fetch	git pull
Conecta al remoto	✓	✓
Descarga cambios	✓	✓
Modifica tus archivos	✗	✓
Cuándo usarlo	Cuando quieres revisar antes de integrar	Cuando quieres actualizar directamente

⚠ ¿Y si git pull genera un conflicto? Si tú y otra persona habéis modificado el mismo archivo, Git no puede fusionarlos solo. Abre el archivo, edita las marcas <<<<<< / ===== / >>>>>>, guarda, haz git add del archivo resuelto y cierra con git commit.

git pull — integrar cambios del remoto

git pull integra un cambio hecho desde la web de GitHub. La salida muestra el fast-forward y las estadísticas del archivo modificado.

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/otro_directorio/mi_proyecto (main)
$ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 246 bytes | 49.00 KiB/s, done.
From https://github.com/aitorventura/mi_proyecto
   d5d381e..9e539b6  main      -> origin/main
Updating d5d381e..9e539b6
Fast-forward
 primer_doc.txt | 1 +
 1 file changed, 1 insertion(+)
```



Ramas en el repositorio remoto

Todo lo que aprendiste sobre ramas funciona igual en el remoto. Una rama local solo existe en tu ordenador hasta que la subes explícitamente.

Publicar una rama en GitHub

```
git switch -c nueva-funcionalidad # crea la rama y cambia a ella
# ... trabajas, haces commits ...
git push -u origin nueva-funcionalidad # la primera vez
git push                               # las siguientes veces
```

Borrar una rama del remoto

```
git push origin --delete nueva-funcionalidad
```

- También puedes borrarla desde la web de GitHub en la sección Branches → icono de papelera.
- Borrar la rama remota no elimina la rama local; hay que borrarla por separado.

Publicar rama con git push -u origin

El aviso [new branch] en la salida de git push confirma que la rama ha llegado al servidor.

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/mi_proyecto (main)
• $ git checkout -b nueva-funcionalidad
Switched to a new branch 'nueva-funcionalidad'

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/mi_proyecto (nueva-funcionalidad)
• $ touch nuevo_archivo.txt

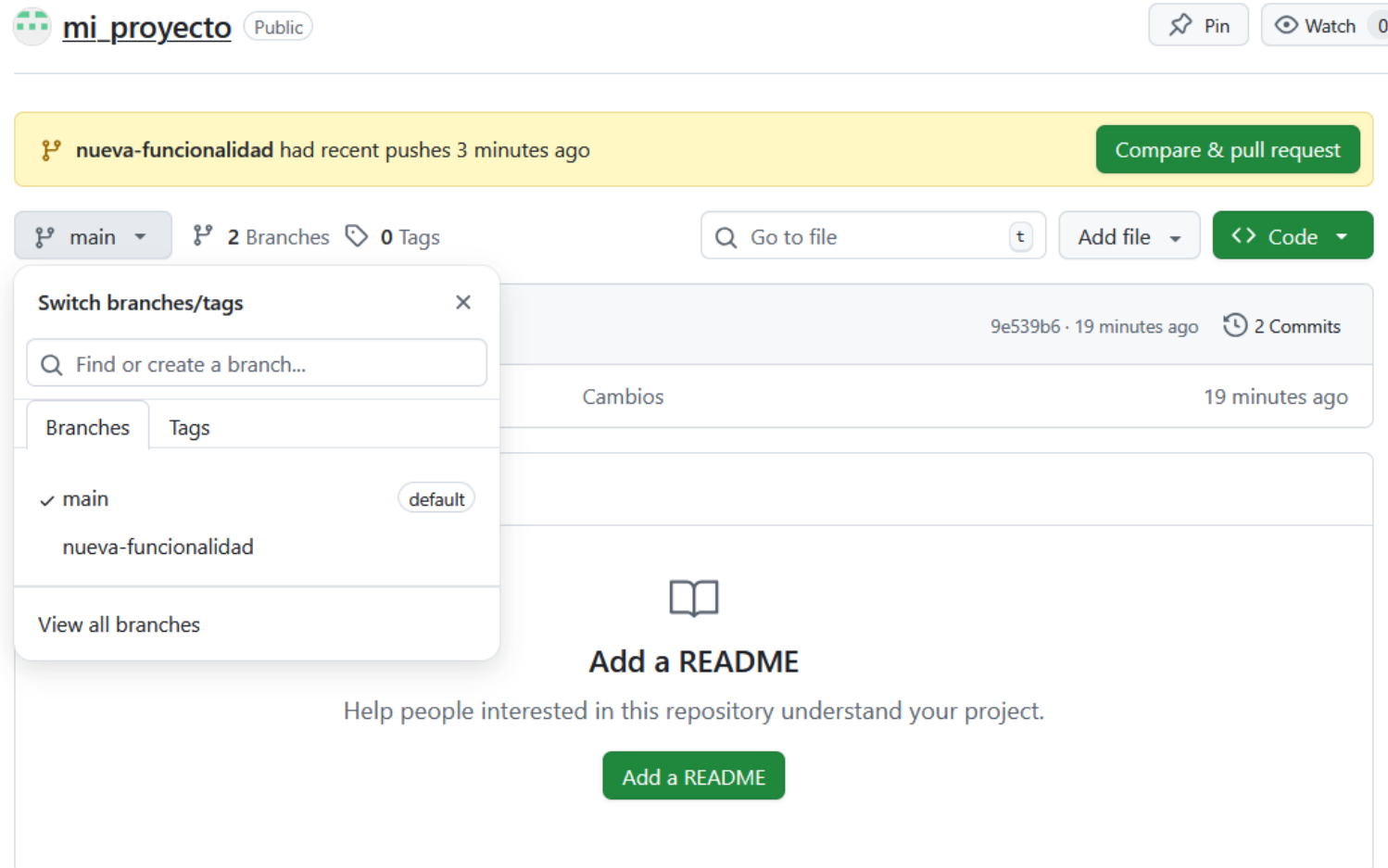
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/mi_proyecto (nueva-funcionalidad)
• $ git add .

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/mi_proyecto (nueva-funcionalidad)
• $ git commit -m "rama"
[nueva-funcionalidad 3a6863a] rama
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 nuevo_archivo.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git/mi_proyecto (nueva-funcionalidad)
• $ git push -u origin nueva-funcionalidad
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 277 bytes | 277.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'nueva-funcionalidad' on GitHub by visiting:
remote:   https://github.com/aitorventura/mi_proyecto/pull/new/nueva-funcionalidad
remote:
To https://github.com/aitorventura/mi_proyecto.git
 * [new branch]      nueva-funcionalidad -> nueva-funcionalidad
branch 'nueva-funcionalidad' set up to track 'origin/nueva-funcionalidad'.
```

Selector de ramas en GitHub

El selector de ramas (arriba a la izquierda del código) muestra todas las ramas subidas. Puedes navegar por ellas o iniciar una Pull Request.



Pull Requests: proponer cambios para revisión

Una Pull Request (PR) es una petición que abres en GitHub para que alguien revise tu rama antes de fusionarla en main. Es el mecanismo central del trabajo colaborativo.

Flujo típico con PRs

- Subes tu rama al remoto: `git push -u origin mi-rama`.
- En GitHub aparece un aviso automático con el botón “Compare & pull request”.
- Escribes un título y una descripción explicando qué has cambiado y por qué.
- Tu compañero revisa los cambios en la pestaña “Files changed” (líneas en verde = añadidas, en rojo = borradas).
- Si todo está bien, se pulsa “Merge pull request” y los cambios pasan a main.
- La rama puede borrarse automáticamente después del merge.

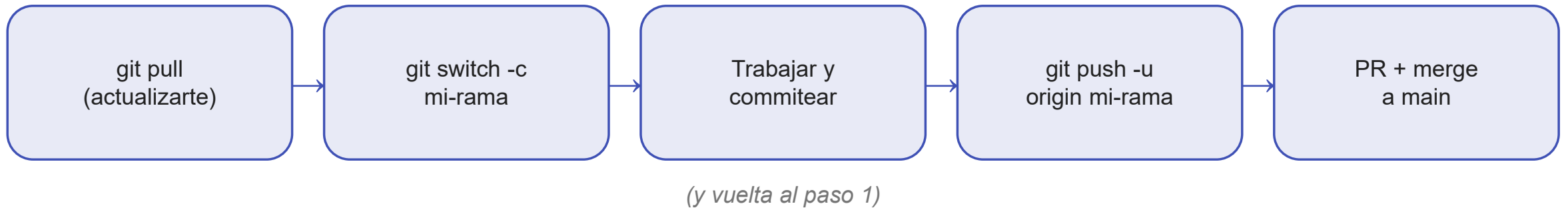
💡 Las PRs no son solo para equipos grandes. Úsalas en proyectos propios para tener historial de decisiones y una forma ordenada de trabajar por funcionalidades.

Pull Request — diagrama de flujo





Flujo de trabajo habitual



Detalle de cada paso

- **git pull**
 - Actualizarte antes de empezar: integra los cambios del remoto en tu rama local.
- **git switch -c mi-rama**
 - Crear una rama para el cambio que vas a hacer; nunca trabajes directo en main.
- **git add . && git commit -m "..."**
 - Commitear con frecuencia con mensajes descriptivos.
- **git push + PR**
 - Publicar la rama y abrir una Pull Request para revisar antes de fusionar.

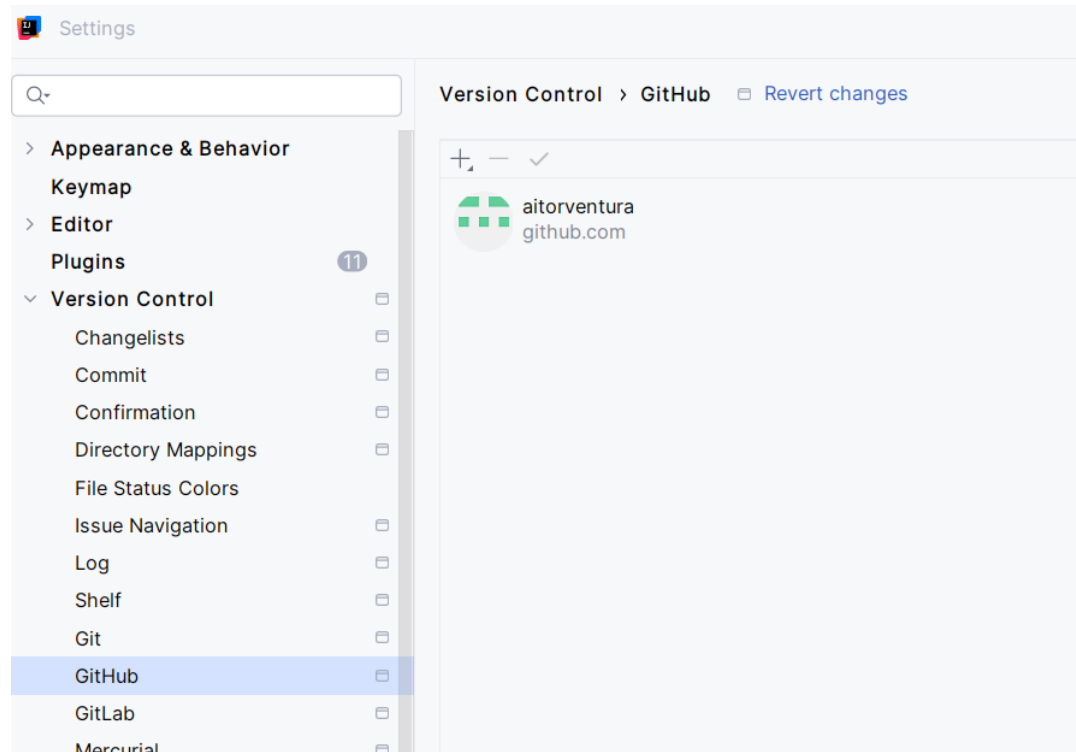


IntelliJ IDEA — Vincular cuenta de GitHub

IntelliJ IDEA incorpora soporte nativo para Git y GitHub: la mayoría de operaciones se pueden hacer desde la interfaz sin usar la terminal.

Enlazar tu cuenta de GitHub

- Abre File → Settings (o Edit → Preferences en Mac).
- Ve a Version Control → GitHub.
- Pulsa + → Log in via GitHub (abre el navegador para autorizar) o introduce tu Token PAT.



IntelliJ — Menú Git: Commit

A) Commit (Ctrl+K)

- Abre un panel donde puedes marcar los archivos a añadir al staging.
- Escribe el mensaje del commit directamente en el panel.
- A la derecha se muestra el diff visual con las líneas añadidas (verde) y borradas (rojo).

Equivale a `git add` + `git commit` en un solo paso visual.

IntelliJ — panel de Commit

The screenshot displays the IntelliJ IDE's Commit interface. On the left, the 'Commit' panel shows a list of 50 files with changes, including `Articulo.java`, `Dado.java`, `Ejercicio8.java` (highlighted as moved), `Ejercicio10.java`, `Ejercicio11.java`, `Empleado.java`, `FechaNacimiento.java`, `Joven.java` (moved), `JuegoDeDados.java`, `Libro.java`, `Main.java`, `MainOperaciones.java`, `Operaciones.java`, and `P01.java`. Below this list, a section titled 'Archivos no subidos' (Files not submitted) is visible. At the bottom of the panel are buttons for 'Commit' and 'Commit and Push...'. The main editor area shows a side-by-side comparison of `Ejercicio8.java`. The left pane shows the current version from the local repository, and the right pane shows the current version from the remote repository. The diff highlights a difference in the package declaration: the local version has `package tema1;` on line 1, while the remote version has `package tema1;` on line 1 and the class definition on line 3. The code in the diff view includes a `public class Ejercicio8` with a `main` method that generates random letters and prints them.

Commit Panel:

- Changes: 50 files
- Files with changes:
 - `Articulo.java` src/main/java/tema5/p05
 - `Dado.java` src/main/java/tema5/p06
 - `Ejercicio8.java` - moved from ..\ src/main/java/tema5/p07
 - `Ejercicio10.java` src/main/java/tema1
 - `Ejercicio11.java` src/main/java/tema1
 - `Empleado.java` src/main/java/refactorizaci...
 - `FechaNacimiento.java` src/main/java/tema5/...
 - `Joven.java` - moved from ..\ src/main/java/tema5/p07
 - `JuegoDeDados.java` src/main/java/tema5/p07
 - `Libro.java` src/main/java/tema7
 - `Main.java` src/main/java/tema7
 - `MainOperaciones.java` src/main/java/tema5/...
 - `misc.xml` .idea
 - `Operaciones.java` src/main/java/tema5/p01
 - `P01.java` src/main/java/tema4
- Amend: 47 added, 3 modified
- Archivos no subidos
- Buttons: Commit, Commit and Push...

Diff View:

Left pane (Local):

```
public class Ejercicio8 {
    public static void main(String[] args) {
        char[] letras = {'a', 'b', 'c', 'd', 'e',
                        'f', 'g', 'h', 'i', 'j', 'k', 'l',
                        'm', 'n', 'ñ', 'o', 'p', 'q', 'r',
                        's', 't', 'u', 'v', 'w', 'x', 'y',
                        'z'};

        for(int i = 0; i < 1000; i++){
            char letraAleatoria = letras[(int)(Math.random() * letras.length)];

            if(letraAleatoria == 'a' || letraAleatoria == 'o' || letraAleatoria == 'u')
                System.out.println("La letra: " + letraAleatoria);
            else {
                System.out.println("La letra: " + letraAleatoria);
            }
        }
    }
}
```

Right pane (Remote):

```
package tema1;

public class Ejercicio8 {
    public static void main(String[] args) {
        char[] letras = {'a', 'b', 'c', 'd', 'e',
                        'f', 'g', 'h', 'i', 'j', 'k', 'l',
                        'm', 'n', 'ñ', 'o', 'p', 'q', 'r',
                        's', 't', 'u', 'v', 'w', 'x', 'y',
                        'z'};

        for(int i = 0; i < 1000; i++){
            char letraAleatoria = letras[(int)(Math.random() * letras.length)];

            if(letraAleatoria == 'a' || letraAleatoria == 'o' || letraAleatoria == 'u')
                System.out.println("La letra: " + letraAleatoria);
            else {
                System.out.println("La letra: " + letraAleatoria);
            }
        }
    }
}
```

IntelliJ — Menú Git: Push y Pull/Fetch

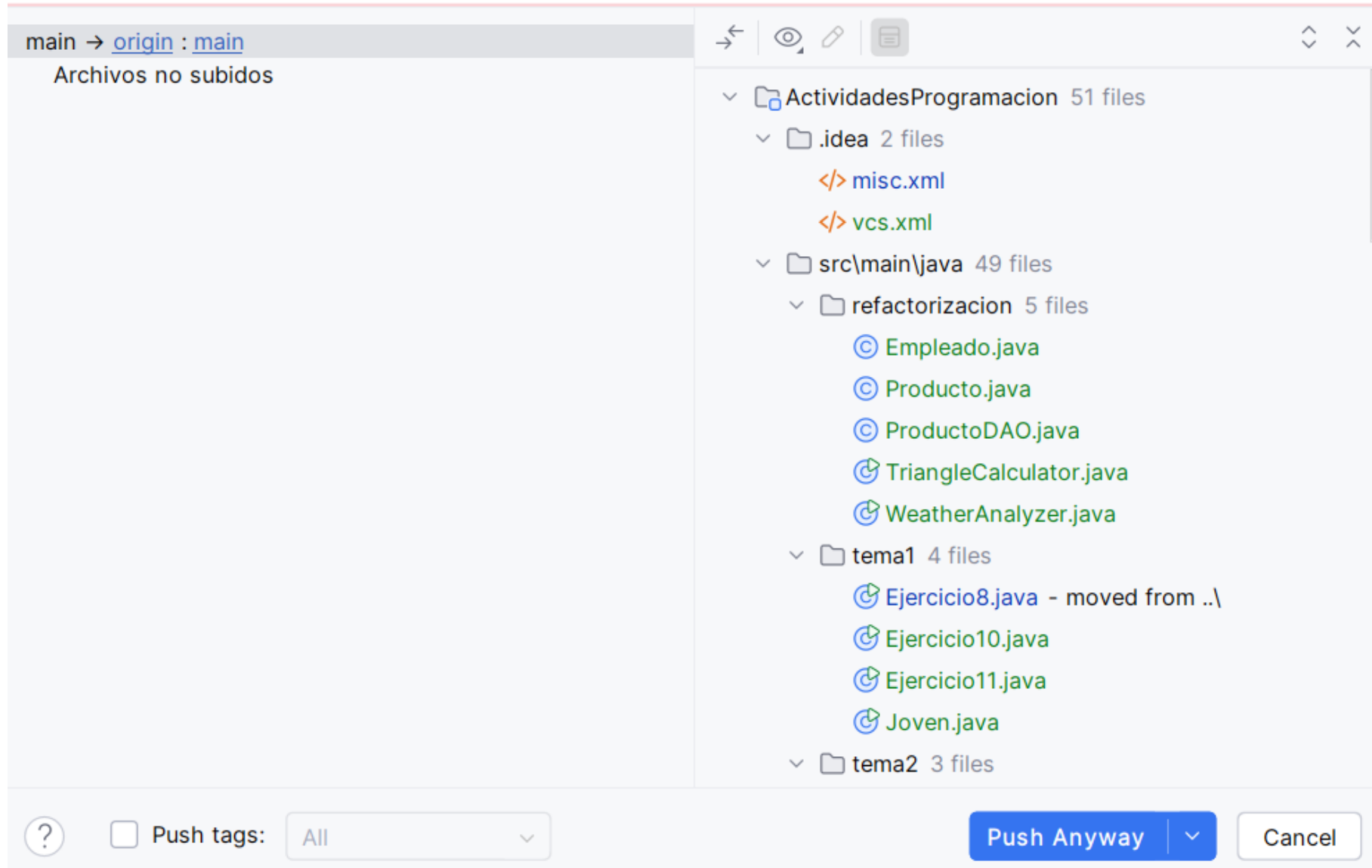
B) Push (Ctrl+Shift+K)

- Muestra los commits pendientes de subir y hacia qué rama de origen van.
- Confirma antes de enviar.

C) Pull / Fetch

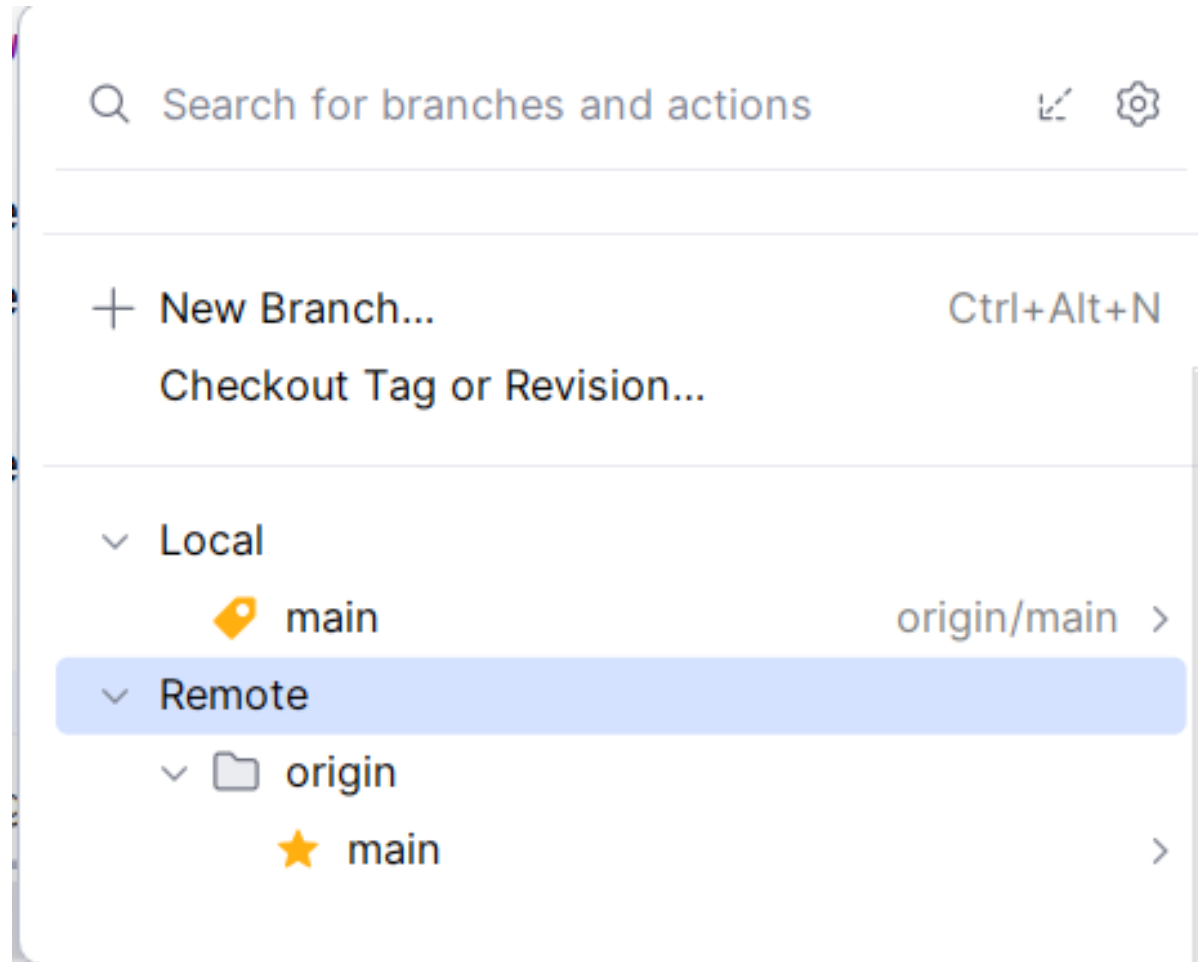
- Descarga e integra los cambios del remoto en tu rama actual.
- La terminal y el IDE son complementarios: la terminal da control total; el IDE acelera las operaciones del día a día.

IntelliJ — diálogo de Push



IntelliJ — Gestión de ramas

Desde Git → Branches puedes ver todas las ramas locales y remotas, crear una nueva rama (equivalente a git switch -c) y cambiar entre ellas.





Ideas clave

Comandos esenciales

- **git remote add origin <url>**
 - Vincula el repo local con el remoto. origin es el alias.
- **git remote -v**
 - Comprueba a qué dirección remota apunta el proyecto.
- **git push -u origin <rama>**
 - Sube la rama la primera vez y la vincula para futuros pushes.
- **git push**
 - Sube los commits de la rama actual al remoto.
- **git clone <url>**
 - Descarga un repositorio completo en una carpeta nueva.
- **git fetch**
 - Descarga el estado del remoto sin modificar tus archivos.
- **git pull**
 - Descarga y aplica los cambios del remoto en tu rama actual.
- **git push origin --delete <rama>**
 - Borra una rama del repositorio remoto.

Una Pull Request es una propuesta de fusión que el equipo revisa antes de integrar en main.